

REST API V1

👤 By Bohdan Mukha🕒 May 14, 2026

Base URL Staging: <https://rythmosservice-stage.intrexis.io>

Authentication for all endpoints is JWT Token that is obtained via client_credentials grant type. Suitable for backend applications.

Swagger API documentation is available at [🌐 Swagger UI](#)

Usage of API

The following flow is a starting point to explore the API.

1. Obtain the JWT token with scopes
2. Obtain available ownership groups /api/v1/ownershipgroups

Obtain available communities /api/v1/communities by providing `ownershipGroupId` as a required query parameter

After step 3, you can explore API based on the scopes you have. Below are examples of integrations

Positioning integration

To build an indoor map of the community, you can use the provided geo JSON files from the /api/communities/{communityId}/floors endpoint. See [🌐 GeoJSON](#) format. This allows you to projectile on the maps solution, we are using the WGS84 coordinate system to position community objects (rooms, elevators, etc), people (residents, employees), and devices

API endpoint /api/v1/devices can be used to obtain information about devices, their current `floorId`, and coordinates within the floor they are located

To subscribe to updates about location, you can use Real Time API, where you as a client subscribe to queue and listen for "location" events.

Alexa integration

To implement Alexa integration it is only needed to provide the correct Rythmos Resident Id to /api/v1/alerting/alexa endpoint. To find resident, /api/v1/residents GET endpoint should be utilized

Residents 2-way sync integration

To implement 2-way sync integration with Rythmos about residents, the next basic things should be implemented.

Obtain information about all residents

/api/v1/residents. This endpoint is requesting `communityId` as a required query parameter and some others are not required. Preferably use pagination wherever possible. Example of pagination query parameters
Page 1: offset=0 limit=10
Page 2: offset=1 limit 10

Create resident

To create a resident, a POST endpoint should be used and all info provided in a simple JSON body.

Fields

- `firstName` - not empty and less or equal to 50 characters
- `middleName` - not empty and less or equal to 20 characters
- `lastName` - not empty and less or equal to 50 characters
- `roomId` - valid room ID. You can obtain and select a valid room ID from /api/v1/communities/{communityId}/rooms endpoint
- `wearStyle` - wearable device wear style. This field is used in Fall Detection functionality and is highly recommended to be set correctly. Possible values, None - 0, Wrist - 1, Necklace - 2
- `wearableDeviceId` - valid wearable device ID. You can obtain and select a valid wearable device ID from the /api/v1/devices endpoint. Wearable Device Type is 3. Also, need to encounter cases where a device is not assigned to anyone already
- `communityId` - valid communityId for which resident should be created

Update resident

All fields should be provided during resident updates. You can't update only specific fields in this endpoint. The scheme is the same as for creating a resident, but needs additionally to provide the Id in the body payload

Manage resident status

To manage resident's status Active and Inactive, you can use /api/v1/residents/{residentId}/status endpoint. Where 1 - Active, 2 - Inactive. Inactive is telling that residents are no longer in the community

Deactivation automatically deactivates all active events for resident and moves the device to the available for assignment

Manage residents devices

Currently, residents are supported to have only one wearable device. There are three different endpoints to manage

Assign device endpoint. This simple endpoint that should be used by every resident was created the first time and the device was assigned later, not on the creation

Unassign device endpoint. This endpoint can be used when a resident no longer uses wearable devices anymore

Swap devices endpoint. This endpoint should be used when residents swap devices because of default reasons like the Device is not working, Device swap, Device is lost. The main difference is that all wearable settings will be copied over from the old wearable to the new wearable. Internally it does unassignment of the old wearable device and assignment of the new one. In the request payload need to provide only the new wearable device ID and reason (see DeviceChangeReason enum)

Staff integration

To read staff (employees) assigned to a community, the Staff endpoint can be used. It returns only staff accounts — resident-linked and relative accounts are excluded. Requires the `api.staff.read` scope.

Obtain information about all staff

/api/v1/staff. This endpoint is requesting `communityId` as a required query parameter and some others are not required. Preferably use pagination wherever possible. Example of pagination query parameters
Page 1: offset=0 limit=10
Page 2: offset=1 limit=10

Query parameters

- `communityId` - valid communityId for which staff should be fetched (required)
- `offset` - pagination offset. Should be passed together with `limit`
- `limit` - page size. Should be passed together with `offset`
- `searchKey` - free-text match on full name
- `isActive` - filter by active status. Omit to get both active and inactive staff. Pass `true` for active only, `false` for inactive only

Response fields per staff item

- `id` - Internal Rythmos user id (string)
- `firstName` - staff first name
- `lastName` - staff last name
- `username` - login username
- `active` - whether the staff member is currently active
- `roleName` - human-readable role (for example "Caregiver", "Nurse", "Facility Administrator")
- `facilityId` - id of the community the staff member is assigned to

Example response

```
1 {
2   "items": [
3     {
4       "id": "42",
5       "firstName": "Maria",
6       "lastName": "Chen",
7       "username": "mchen",
8       "active": true,
9       "roleName": "Caregiver",
10      "facilityId": "95"
11    },
12    {
13      "id": "50",
14      "firstName": "James",
15      "lastName": "Decker",
16      "username": "jakafor",
17      "active": true,
18      "roleName": "Nurse",
19      "facilityId": "95"
20    }
21  ],
22  "total": 2
23 }
```

Staff events

When a staff member accepts or deactivates an alert from the Rythmos backend or apps, the Real Time API alert event carries an additional `data.staffId` field containing the internal Rythmos user id of the staff member who performed the action. To resolve `staffId` to a full staff profile, use the `GET /api/v1/staff` endpoint above.

Two cases carry `data.staffId`:

1. **Deactivation by staff** — existing deactivation events (enum values 2081-2026, suffix `DeactivationFromRythmos`) include `data.staffId` when the facility is enrolled.
2. **Accept by staff** — new event emitted when a staff member taps "Accept" on an incoming alert. Uses the new enum values 3001-3026 (suffix `AcceptedByStaff`). The accept enum mirrors the deactivation enum one-for-one with a +1000 offset (for example, `ButtonPressDeactivationFromRythmos = 2081` ↔ `ButtonPressAcceptedByStaff = 3001`).

Consumers distinguish accept from deactivate by the `eventType` range:

- 2081-2026 → deactivation by staff
- 3001-3026 → accept by staff

Both the accept and deactivation events share the same `id` as the original activation (the pendant event id prefixed with `P`, for example "P1234567"), so clients can correlate a full alert lifecycle: activation → accept → deactivation.

Example accept event

```
1 {
2   "type": "alert",
3   "event": {
4     "id": "P1234567",
5     "eventType": 3001,
6     "communityId": "95",
7     "deviceId": "SH-A1B2C3D4",
8     "timestamp": "1775999400000",
9     "nearestAnchorId": "10042",
10    "location": "Apt 204 - Living Room",
11    "roomId": "15",
12    "data": {
13      "rythmosResidentId": "789",
14      "staffId": "42",
15      "deactivationOptionId": 101,
16      "deactivationOptionName": "True Alert"
17    }
18  }
19 }
```

Example deactivation event with staff info

```
1 {
2   "type": "alert",
3   "event": {
4     "id": "P1234567",
5     "eventType": 2081,
6     "communityId": "95",
7     "deviceId": "SH-A1B2C3D4",
8     "timestamp": "1775999400000",
9     "nearestAnchorId": "10042",
10    "location": "Apt 204 - Living Room",
11    "roomId": "15",
12    "data": {
13      "rythmosResidentId": "789",
14      "staffId": "42",
15      "deactivationOptionId": 101,
16      "deactivationOptionName": "True Alert"
17    }
18  }
19 }
```

Notes

- `data.staffId` is populated only for facilities explicitly enrolled in the staff id stream. Facilities that are not enrolled continue to receive events in the existing shape, without a `data.staffId` field and without any accept events.
- Consumers that don't know about `data.staffId` can safely ignore the field — it is additive, not breaking.
- Accept events (3001-3026) are only emitted for enrolled facilities. Consumers that don't recognize these enum values should treat them as unknown event types and ignore them.

For the full Service Bus envelope structure, Java integration example, and complete `AlertType` enum (all event types, not just staff), see the [Real Time API V1](#) page.

Postman collection

It contains requests to the REST API. Before use, you need to

1. Set client id and secret into variables
2. Get token
3. Set token globally on the collection level to Authorization Token

Enums that are used in the REST API

```
1 enum DeviceChangeReason
2 {
3   DeviceLost = 1,
4   DeviceNotWorking = 2,
5   DeviceSwap = 3
6 }
```

```
1 enum DeviceTypes
2 {
3   Unknown = 0,
4   Gateway = 1,
5   Anchor = 2,
6   Wearable = 3,
7   SmokeDetector = 4,
8   Lock = 5,
9   Badge = 6,
10  Reader = 7,
11  PullCord = 8,
12  WindowSensor = 9,
13  DoorSensor = 10,
14  PresenceDetectionSensor = 11,
15  DoorBell = 12,
16  HandWashing = 13,
17  DoorPositionSwitch = 14,
18  Asxcting = 15,
19  ThermalSensor = 16,
20  IncontinenceSensor = 17
21 }
```

```
1 enum NetworkStatuses
2 {
3   Offline = 0,
4   Online = 1
5 }
```

```
1 enum ResidentStatus
2 {
3   Active = 1,
4   NotActive = 2
5 }
```

```
1 enum ResidentWearStyle
2 {
3   None = 0,
4   Wrist = 1,
5   Necklace = 2
6 }
```

Near each type in the comment if that's supported for emulation

```
1 public enum AlertType
2 {
3   Unknown = 0,
4   ButtonPressActivation = 1, // Supported
5   ButtonPressDeactivation = 2, // Supported
6   SoftFallActivation = 3, // Supported
7   HardFallActivation = 4,
8   FallDeactivation = 5, // Supported
9   WanderActivation = 6, // Supported
10  PullCordActivation = 7, // Supported
11  PullCordDeactivation = 8, // Supported
12  WindowSensorOpen = 9, // Supported
13  WindowSensorClose = 10, // Supported
14  DoorSensorOpen = 11, // Supported
15  DoorSensorClose = 12, // Supported
16  SmokeDetectorActivation = 13, // Supported
17  SmokeDetectorDeactivation = 14, // Supported
18  DoorBellActivation = 15, // Supported
19  DoorBellDeactivation = 16, // Supported
20  MotionSensorActivation = 17, // Supported
21  MotionSensorDeactivation = 18, // Supported
22  PresenceDetectionSensorStart = 20, // Supported
23  AlexahelpRequestActivation = 23, // Supported
24  LoiteringActivation = 24, // Supported
25  LoiteringDeactivation = 25, // Supported
26  GeofenceKeepOut = 26,
27  DoorBreached = 28, // Supported
28  DoorPropped = 29, // Supported
29  DoorOpened = 30, // Supported
30  DoorClosed = 31, // Supported
31  MeshConnect = 32,
32  MeshConnect = 33,
33  GeofenceKeepIn = 34,
34  DoorBreachedAttempt = 36, // Supported
35  OutOfBedAlert = 37, // Supported
36  CellularButtonPressActivation = 1001, // Supported
37  CellularButtonPressDeactivation = 1002, // Supported
38  CellularSoftFallActivation = 1003, // Supported
39  CellularSoftFallDeactivation = 1004, // Supported
40  CellularWanderActivation = 1007,
41
42  // Deactivation by staff (from Rythmos backend feature or app)
43  FallDeactivationFromRythmos = 2001,
44  WanderDeactivationFromRythmos = 2002,
45  PullCordDeactivationFromRythmos = 2003,
46  WindowSensorOpenedDeactivationFromRythmos = 2004,
47  WindowSensorOpenedDeactivationFromRythmos = 2005,
48  WindowSensorClosedDeactivationFromRythmos = 2006,
49  DoorSensorOpenedDeactivationFromRythmos = 2007,
50  DoorSensorClosedDeactivationFromRythmos = 2008,
51  SmokeDetectorDeactivationFromRythmos = 2009,
52  DoorBellDeactivationFromRythmos = 2010,
53  MotionSensorDeactivationFromRythmos = 2011,
54  PresenceDetectionSensorMotionDeactivation = 2012,
55  AlexahelpRequestDeactivationFromRythmos = 2013,
56  GeofenceDeactivationFromRythmos = 2015,
57  DoorBreachedDeactivationFromRythmos = 2017,
58  DoorProppedDeactivationFromRythmos = 2018,
59  DoorClosedDeactivationFromRythmos = 2019,
60  DoorClosedDeactivationFromRythmos = 2020,
61  DoorBreachedAttemptDeactivationFromRythmos = 2022,
62  OutOfBedAlertDeactivationFromRythmos = 2023,
63  CellularButtonPressDeactivationFromRythmos = 2024,
64  CellularFallDeactivationFromRythmos = 2025,
65  CellularWanderDeactivationFromRythmos = 2026,
66
67  // Accept by staff - mirrors the deactivation range above with a +1000 offset
68  ButtonPressAcceptedByStaff = 3001,
69  FallAcceptedByStaff = 3002,
70  WanderAcceptedByStaff = 3003,
71  PullCordAcceptedByStaff = 3004,
72  WindowSensorOpenedAcceptedByStaff = 3005,
73  WindowSensorClosedAcceptedByStaff = 3006,
74  DoorSensorOpenedAcceptedByStaff = 3007,
75  DoorSensorClosedAcceptedByStaff = 3008,
76  SmokeDetectorAcceptedByStaff = 3009,
77  DoorBellAcceptedByStaff = 3010,
78  MotionSensorAcceptedByStaff = 3011,
79  PresenceDetectionSensorMotionAcceptedByStaff = 3012,
80  AlexahelpRequestAcceptedByStaff = 3013,
81  LoiteringAcceptedByStaff = 3014,
82  GeofenceAcceptedByStaff = 3015,
83  WearableKeenovedAcceptedByStaff = 3016,
84  DoorProppedAcceptedByStaff = 3017,
85  DoorOpenedAcceptedByStaff = 3018,
86  DoorClosedAcceptedByStaff = 3019,
87  IncontinenceAlertAcceptedByStaff = 3021,
88  DoorBreachedAttemptAcceptedByStaff = 3022,
89  OutOfBedAlertAcceptedByStaff = 3023,
90  CellularButtonPressAcceptedByStaff = 3024,
91  CellularFallAcceptedByStaff = 3025,
92  CellularWanderAcceptedByStaff = 3026,
93
94 }
95 }
```